In which you will explore some of the low-level details of C programming that you've been able to ignore when using higher-level languages.

Purpose and outcomes

The goal of this lab is to explore how programs work when you don't get as much help from the compiler or run-time as you would with a higher-level language like C++, Go, Java or Rust. By the end of this lab you should be able to:

1. manage memory manually,
2. use (and misuse!) C-style strings,
3. use `libc` functions for I/O,
4. use compiler flags, manual pages and inspection tools such as `nm(1)` and `objdump(1)`,
5. discuss symbol naming in C vs C++ and other languages, and
6. write and use C macros.

Preparation

Prerequisites

Ensure that your LabNet account is in working order. You may also like to identify a lab partner in advance of the lab, but if you don't have a partner, you can pair with someone at the beginning of the lab.

Procedure

Complete the steps described below, recording each step (including the commands you run and their key outputs) and answering all questions.

Symbols and polymorphism

Let's consider the implications of C++ function (and operator) overloading, which C doesn't have, on the artifacts produced by the compilation process.

1. Write two C++ functions with the following signatures:

```
int add(int, int);
float add(float, float);
```

Save them in a file called `cpp-math.cc`. Compile this C++ file to an object (`.o`) file and inspect its symbols with the `nm` tool. What do you observe? You may also find it helpful to *demangle* these symbols by either passing the `--demangle` argument to `nm` or by piping the output of `nm` to `c+filt` (i.e., `nm cpp-math.o | c+filt`).

2. Save the following to a header file `cpp-math.h`:

```
extern "C" {

int add(int, int);

}
```

`#include` this header file from `cpp-math.cc` and re-compile the object file `cpp-math.o`. How are the symbols in this file different now?

3. Save your function implementations to a new file `c-math.c`. Attempt to compile this to an object file. What do you observe? What can you infer from this observation (and those above) about an important difference between C and C++?

4. Comment out the `float` version of `add` in `c-math.c` and compile to `c-math.o`. Compare the implementations of `int add(int, int)` found in `c-math.o` and `cpp-math.o` using `objdump` and `diff`, `colordiff` or a GUI tool such as Meld. (extra: you may also want to use Pygments to colourize the output of `objdump`, e.g., `objdump -dS c-math.o | pygmentize -l c-objdump`)

C-style strings

A C-style string is an array of characters (i.e., `char[]` or `char*`), terminated by a `NUL` character (i.e., `(char) 0`). This is a somewhat less sophisticated representation than, say, a C++ `std::string`, which is an object containing a character array *and* a length. This simplistic representation leads to all sorts of terrible scrapes, but those details are beyond the scope of this course!

1. In a new C file containing a `main` function, use the C library function `fwrite(3)` to write the string `"Operating systems rule"` to `stdout`. Use a debugger (`gdb` or `lldb`) to identify the address of the string literal `"Operating systems rule"`.

2. Create a `char` array initialized with your initials (e.g., I'd use `"JRA"`). Use `strlen(3)` to find the length of this C-style string. Use a debugger to identify the characters stored in this array

(hint: for GDB, use `x/8x`; for LLDB, use `mem read`).

3. Use the `strcpy(3)` function to copy your first name into this array. What does `strlen(3)` now report? How does this compare to the number of bytes that was allocated for the array? Use a debugger to show the bytes now held in the array.

I/O with `libc`

C doesn't have overloaded operators like C++:

```
template<typename T> ostream& operator << (ostream&, T);
```

It also doesn't use dynamic typing like Python:

```
def output_something(*args):
    for a in args:
        print(type(a))
        print(a)
```

Instead, we can use the *formatted print* function `printf(3)`. `printf(3)` and its variants use a *format string* to specify what types of values are being printed, followed by the values themselves. This uses a C function technique called *varargs* (variadic arguments), which allows you to pass an arbitrary number of arguments to a function, and the the function takes responsibility for handling **whatever types of values** are passed.

1. Print out your favourite integer using `printf(3)` with the format string `"My favourite integer is: %d"`, where `%d` signifies a decimal representation of an integer.

2. Using the manual page for `printf(3)`, print out a floating-point number, a hex-formatted integer and a pointer to something in your program.

3. Use `sprintf(3)` to generate a string describing the string containing your name (e.g., `"my name is 'Jonathan Anderson', which is 17 characters long, and it is located at address 0x7fff12345678"`). Write this string to the standard output stream, first using the standard C library's `fwrite(3)` and its `stdout` stream, then using the Unix `write(2)` system call and the `STDOUT_FILENO` file descriptor.

Memory management

C doesn't provide automatic garbage collection like many modern high-level languages, or even scopable memory owners like C++ smart pointers or a Rust `Box`. Instead, the developer must take responsibility for memory allocation with `malloc(3)` (and friends) and de-allocation with `free(3)`.

1. Use `malloc(3)` to allocate space to store one integer and print its address. Run the program. What do you observe?

2. Re-compile the program using the compiler flag `fsanitize=address`. Now what do you observe when you run the program? Explain the difference.

3. Use both `malloc(3)` and `calloc(3)` to allocate arrays of one thousand integers. Demonstrate, using a debugger, that the allocated arrays are different in an important way that is particular to `calloc(3)`.

4. Write a program that correctly allocates **and** de-allocates space to store an array of 100 bytes. What is the value of the array pointer after de-allocation?

5. What happens when you call `free(3)` on an already-deallocated array?

6. What happens when you call `free(3)` on a `NULL` pointer?

Macros

Many languages have some form of *metaprogramming:* a way to program around the program. In C++, templates and other compile-time tricks allow for complex computation to be performed at compile time rather than run time. Python and other dynamic languages provide functions like `eval()` that allow a program to be included in a program without that source code being available when the program's source code is inspected. Rust includes *procedural macros*, which are a much cleaner form of a dirty hack that C includes: string-based macros.

`#define` allows you to replace an identifier with an almost-arbitrary string that contains C code. There are also function-like macros, but we won't get into those today.

1. Write a C file (no `main` function is required) that includes the macro definition `#define FOO "this is the definition of foo"`. Use `cc -E` to *pre-process* this file, outputting a more verbose C file. Does this more verbose file contain the string `FOO`?

2. Now write a function that uses `FOO`, e.g., indexing within it. What changes when you pre-process this file?

3. Now add `#include <stdio.h>` and modify your code to print `FOO`. What changes in the pre-processed output?

4. Find the definition of `NULL` within `/usr/src/linux-headers-XXXX/include/linux/stddef.h`. How is this definition of `NULL` different from C++'s `nullptr`?